

XSKILL

XSKILL: Continual Learning from Experience and Skills in Multimodal Agents

<https://arxiv.org/pdf/2603.12056> (Guanyu Jiang Zhaochen Su *HKUST*)

1.motivation

(1)工具使用效率低

简单问题上耗费过多步骤，面对复杂查询时却又无法开展足够深入的多轮探索

例：

codex写入命令行指令时使用的是老版本的，与当下windows系统不兼容

Skill（技能）提供结构化工作流和工具模板，告诉智能体：

应该调用哪些工具、按照什么顺序调用、具体怎样调用

(2)工具编排僵化

不能根据具体任务对当下步骤进行灵活的工具选择

例：旋转图片 → 裁剪目标 → 反向图片搜索 → 浏览网页

experience（经验）保存情境相关的“条件—动作”知识，告诉智能体：

在当前情况下，应该选择或调整哪个工具

(3)智能体在提取、检索和使用记忆时，主要依据文本文字，没有把当时的图像状态纳入记忆系统

2.方法

Skill 示例：VisualLogicArchitect.md (v20.0.0)

元数据：

- 名称：VisualLogicArchitect
- 描述：一个统一框架，覆盖多领域视觉分析——定量文档审计（财务/TAT）、网络与几何路径搜索、科学-临床诊断（NMR/生存曲线）、战术时空场景分析（体育/行情图）

- 版本：20.0.0

适用场景：

- 定量与文档审计：验证财务表格（桑基图、损益表）、收据、临床报告的周转时间（TAT）与认证合规性
- 网络与几何系统：求解电路、公交线路规划、分析径向仪表盘或杠杆物理

- 科学与临床诊断：分析 NMR 光谱、基因组图、Kaplan-Meier 生存曲线
 - 战术与时空场景：判断比赛结果（体育）、市场定位、“假设分析”型仪表盘重排
- 策略总览：**

1. 归一化与朝向检测：检测反转的文字或坐标轴，用 Python 翻转/旋转至可读
2. 时空重排序：对运动或流向，重建“刚刚之前”和“刚刚之后”的状态，用阴影位置锚定真实位置
3. 分割与 ROI 普查：隔离感兴趣区域，对密集标记做系统性扫描防止重复计数
4. 逻辑校验：应用领域规则（欧姆定律、NMR DEPT 逻辑、市场跨度规则、临床认证标准）

工作流 1：财务、流程与文档审计

1. 数据普查与提取：定位“总计”节点或“收据总额”，识别子贡献者和“排除”节点
2. 相关基数计算：减去排除值或累加特定子类别来计算分母
3. 指标排序与综合：按增长率/值排序得出前 N 名；对 TAT 分析，搜索认证基准并从原始行项目重新计算总额

工作流 2：几何、径向与网络系统

4. 坐标与地标映射：确定原点和坐标轴方向（反转系统中“数学上最高”可能在物理上位于底部）
5. 路径效率审计：将地标映射到最近站点，比较枢纽路线（1 次换乘）与跨城路线（2+ 次换乘），若 2 次换乘路线站数少 $\geq 25\%$ 则可能更快
6. 几何提取：对径向路径，从圆心向外锚定以防跳线；计算相对于支点的力臂

工具模板 (Python)：

python

```
from PIL import Image, ImageOps, ImageEnhance
from datetime import datetime

def process_visual(img_path, rotate=0, flip_h=False, crop_norm=None):
    img = Image.open(img_path)
    if flip_h: img = ImageOps.mirror(img)
    if rotate: img = img.rotate(rotate, expand=True)
    if crop_norm: # [ymin, xmin, ymax, xmax] 0-1000
        w, h = img.size
        img = img.crop((crop_norm[1]*w/1000, crop_norm[0]*h/1000,
                        crop_norm[3]*w/1000, crop_norm[2]*h/1000))
    return img

def calculate_duration(start_str, end_str, fmt="%d/%m/%Y %H:%M"):
    start = datetime.strptime(start_str, fmt)
    end = datetime.strptime(end_str, fmt)
    delta = end - start
    return delta.total_seconds() / 3600 # 返回小时数
```

Experience 示例

E0: 在复杂结构化问题中，识别"零影响"区域——数据不影响最终结果的区域。识别出这些区域（如块三角矩阵中的"零区"）可以忽略无关变量或噪声，简化推理并减少错误

E2: 要确认某个角色确实不存在，需对"阴影陷阱"（暗色或杂乱的背景区域）专门做亮度和对比度增强。只有在对这些增强区域做高倍扫描后仍找不到，才能宣称角色缺失——小轮廓或背景客串在标准视图中很容易被漏掉

E5: 在解读趋势或识别"当前"价格之前，先用坐标轴标签验证时间线。如果日期从左到右递减，说明图像是镜像的，需要水平翻转

E22: 对旋转、镜像或变形的布局，用代码（ImageOps.mirror、单应矩阵）归一化以恢复文字流向、防止字符幻觉。用参考点验证可读性。变换后重新验证坐标系和原点偏移

E46: 做基于视觉标记的分类时，实施"必要特征检查"。在定稿类别前，验证所有必需的次要特征是否存在、矛盾特征是否不存在。这种交叉验证可防止单个视觉幻觉或误识导致整个推理链的级联失败

(1) 获取Skill和Experience

假设训练集里有一道题：给一张倒置的图片，问"图中招牌写了什么？"，正确答案是"HELLO"

第一步：跑 Rollout (2.2.1 前半)

（代码执行、网页搜索、图像搜索、网页访问）+MLLM

MLLM_exec 对这道题独立跑 4 次（N=4），得到 4 条流程：

- 流程 1：直接 OCR → 读出乱码 → 答错
- 流程 2：直接 OCR → 读出乱码 → 答错
- 流程 3：先旋转图片 → OCR → 读出"HELLO" → 答对
- 流程 4：先增强对比度 → OCR → 读出乱码 → 再旋转 → OCR → 答对

第二步：Rollout 摘要 (2.2.1 后半)

把 4 条轨迹连同原图、题目、正确答案一起丢给 MLLM_kb，让它出摘要。MLLM_kb 会看到图片确实是倒的，然后总结出：

- 1、2 失败原因：没发现图是倒的就直接 OCR
- 3 成功关键：先旋转再 OCR
- 4 成功但绕了弯：增强对比度没用，最后还是靠旋转
同时抽出技能片段 ΔK ：一个 workflow "检测朝向 → 旋转 → OCR"，一个工具模板 `img.rotate([angle])`

第三步：跨 Rollout 批判 (2.2.2)

MLLM_kb 对比成功轨迹（3、4）和失败轨迹（1、2），问：成功的做了什么、失败的没做什么？结论是：**关键差异在于是否检查了图片朝向**。于是输出一条经验更新操作：

（add, "OCR 前先检查图像是否需要旋转，倒置或镜像的文字会导致识别失败"）

第四步：知识整合 (2.2.3)

- 经验管理器：这条新经验拿去和经验库里已有的条目算余弦相似度。如果已经有一条类似的"处理图像前检查朝向"，就合并成一条更泛化的；如果没有，直接加入（相似度 >0.70 就合并，超 120 条就裁剪）
- 技能管理器：工作流片段和工具模板合并进全局技能文档。如果文档太长了（超过 1000 词），MLLM_kb 自动精炼，删掉过于具体的细节

(2) 用Skill和Experience做新题

新题：一张倒置的图片，角落有两个很小的吉祥物，问"它们的原型是什么？"

第一步：任务分解检索

MLLM_kb 看了题目和图片，拆成三个子任务：

- 子任务1: "图像朝向异常的处理" → 检索到经验 E2（识别失败时检查是否需要翻转）、E5（检查坐标轴/图像方向）
- 子任务2: "小物体的识别" → 检索到经验 E22（用代码裁剪放大）
- 子任务3: "未知物体的知识获取" → 检索到经验 E3（遇到未知视觉知识时用图像搜索）
如果不拆分，直接拿"吉祥物原型是什么"去检索，大概率只命中子任务3相关的经验，漏掉前两个

第二步：任务适配（经验改写 + 技能适配）

MLLM_kb 看了当前这张图，发现图确实是倒的、吉祥物确实很小，于是：

- E2 从"识别失败时检查图像是否需要翻转" → 改写成"这张图是倒置的，先用 `image.rotate(180)` 翻转"
- E22 从"用代码裁剪放大" → 改写成"吉祥物在左上角且很小，用 `image.crop` 裁剪该区域"
- E3 保持不变，仍然适用
- 技能文档裁掉财务审计、电路分析等无关工作流，只保留"检测朝向→旋转→裁剪→搜索"这条工作流和对应的代码模板

第三步：注入执行

把改写后的经验和裁剪后的技能拼进 MLLM_exec 的 prompt

MLLM_exec 看到这些参考后，执行：旋转 → 裁剪 → 图像搜索裁剪图 → 从网页读到答案是 dolphin。执行完后记录下实际用了 E2、E22、E3 和那条工作流

3.实验

四个工具：代码执行、网页搜索、图像搜索、网页访问

数据集：

第一类：**视觉智能体工具使用**，聚焦于结合多工具操作的视觉推理

VisualToolBench 和 TIR-Bench，它们要求智能体处理视觉输入并精确执行图像分析工具

第二类：信息检索和网页浏览实现的**多模态搜索**

采用 MMSearch-Plus 和 MMBrowseComp 来考验智能体在异构文本和视觉来源上进行搜索和推理的能力

第三类：

AgentVista，这是一个超高难度的**综合性**基准，兼具工具使用和复杂搜索任务

随机采样 100 个任务来构建训练集以用于经验累积，剩余任务保留用于评测

1. VisualToolBench

考的是"看图+用多种工具联合解题"

给一张财务报表的图片，算某个指标；或者给一张体育比赛照片，判断比赛结果

agent 需要写 Python 代码处理图像、搜网页查信息、组合多个工具才能做

论文从中选了单轮混合工具推理的子集，100 题训练，214 题测试

2. TIR-Bench

也是视觉推理，但更细分

原始数据集有 13 个类别，论文筛了 5 个和工具集兼容的：RefCoCo（给一段描述定位图中物体）、迷宫（空间推理）、仪器（物体检测）、OCR（文字识别）、对比度（视觉对比分析）

只配了代码执行一个工具，没有搜索。100 题训练，200 题测试

3. MMSearch-Plus

考的是多模态搜索。

给一张图加一个问题，需要结合网页搜索和图像搜索去找答案

比如给一张建筑照片问"这是哪个城市的什么建筑"，agent 得先图像搜索找到相似图片，再访问网页读详情

四个工具全用上。100 题训练，211 题测试

4. MMBrowseComp

也是多模态搜索方向，但更侧重网页浏览理解

只有 130 题，数据量太少没法拿来训练。论文把它纯做测试集，用 MMSearch-Plus 攒的知识来做它，专门测跨任务迁移能力

5. AgentVista

最难的一个，综合性基准

同时包含工具使用和复杂搜索，任务场景更接近真实世界

比如可能需要你先用代码处理图片、再搜索、再访问网页、再写代码分析搜到的结果，多步串联。100 题训练，109 题测试

三个基线：

AWM 怎么做

那条成功轨迹提取出一个工作流："Step1: 检查图像朝向 → Step2: 裁剪目标区域 → Step3: 搜索裁剪后的图像"

推理时拿题目去检索，如果这条工作流恰好是最相似的就能检索到，直接注入 prompt。问题是：不看图，不知道当前这张图到底是不是倒的，工作流也不会根据当前图像做任何调整

DC 怎么做

训练题做完后，备忘录可能更新成："处理图像任务时注意检查朝向，小物体需要裁剪放大后再搜索"

推理时把整份 200 词的备忘录塞进 prompt。

问题：备忘录是所有题累积覆盖更新的，随着新题不断进来，这条关于朝向的知识可能被挤掉或稀释。而且也不看图，不做适配

Agent-KB 怎么做

最接近 XSkill，有结构化的知识库，分成规划指导和经验反馈两部分

知识库存了一条经验"图像倒置时先旋转再处理"

用混合检索（TF-IDF + 语义相似度）找到这条，注入 prompt

比 AWM 和 DC 好一些，但还是纯文本检索——它不看图，不知道当前图确实是倒的，也不会把"旋转"具体化成"旋转180度"

Table 2. Main results of performance comparison (%) between XSKILL and baselines. We report the Average@4 and Pass@4 over 4 independent rollouts. **Bold** indicates the best performance.

Model	Methods	VisualToolBench		TIR-Bench		MMSearch-Plus		AgentVista		Avg	
		Average@4	Pass@4	Average@4	Pass@4	Average@4	Pass@4	Average@4	Pass@4	Average@4	Pass@4
Gemini-2.5-Pro	No Tools	20.91	28.97	21.37	40.00	10.43	19.43	17.89	28.44	17.65	29.21
	w/ Tools	25.35	40.65	28.38	54.00	21.56	35.55	20.18	33.94	23.87	41.04
	AWM	25.93	39.25	29.75	53.50	20.85	36.97	19.72	32.11	24.06	40.46
	DC	24.77	37.38	27.62	51.00	24.64	40.76	21.79	35.78	24.71	41.23
	Agent-KB	26.75	41.12	29.13	52.50	23.22	37.91	20.87	33.94	24.99	41.37
	XSkill	30.49	46.73	33.12	58.00	27.96	44.08	22.94	34.86	28.63	45.92
Gemini-3-Flash	No Tools	25.12	36.92	28.50	54.00	16.47	24.64	18.35	29.36	22.11	36.23
	w/ Tools	41.94	60.75	32.37	58.50	39.57	53.55	20.64	39.45	33.63	53.06
	AWM	41.94	59.35	34.25	62.50	43.36	54.98	19.50	37.61	34.76	53.61
	DC	41.70	59.81	33.75	59.00	40.28	55.45	20.18	36.70	33.98	52.74
	Agent-KB	41.75	61.21	36.62	62.00	39.81	53.08	21.33	38.53	34.88	53.71
	XSkill	46.50	64.02	47.75	75.00	43.72	56.40	23.39	40.37	40.34	58.95
GPT-5-mini	No Tools	13.90	22.90	20.00	46.50	3.08	6.64	18.58	28.44	13.89	26.12
	w/ Tools	24.30	37.85	23.50	50.50	14.22	20.38	20.41	35.78	20.61	36.13
	AWM	23.25	34.58	24.25	53.00	14.81	20.85	19.04	34.86	20.34	35.82
	DC	23.83	35.05	24.50	53.50	13.74	18.48	21.10	36.70	20.79	35.93
	Agent-KB	24.77	38.32	25.13	53.00	13.63	19.91	19.95	34.86	20.87	36.52
	XSkill	24.53	37.85	28.25	56.00	16.11	23.22	23.85	38.53	23.19	38.90
o4-mini	No Tools	14.72	27.57	20.13	45.00	5.92	12.32	19.04	25.69	14.95	27.65
	w/ Tools	19.63	34.11	24.62	49.50	15.88	21.80	18.12	29.36	19.56	33.69
	AWM	21.14	36.92	25.25	51.00	16.94	22.75	19.95	29.36	20.82	35.01
	DC	20.44	33.64	25.50	52.00	14.57	22.27	18.81	28.44	19.83	34.09
	Agent-KB	22.78	36.92	24.75	52.50	15.17	20.38	19.50	31.19	20.55	35.25
	XSkill	25.00	41.12	30.25	57.50	17.30	23.70	22.32	33.94	23.72	39.07

Average@4：4 次里做对几次，算比例。比如做对 3 次，这道题的 Average@4 就是 75%。所有题的比例取平均，就是最终指标。衡量的是**可靠性**

Pass@4：4 次里只要有 1 次做对，这道题就算过。衡量的是**能力上限**

- Gemini-2.5-Pro 使用自己产生的推理轨迹积累 Experience 和 Skill；
- Gemini-3-Flash 同样使用自己的轨迹积累知识；
- GPT-5-mini 不自行积累知识，而是直接使用 Gemini-3-Flash 建立的知识库；
- o4-mini 同样直接使用 Gemini-3-Flash 的知识库

后两个实验用于检验：一个模型积累的 Experience 和 Skill，能否迁移给另一个模型使用

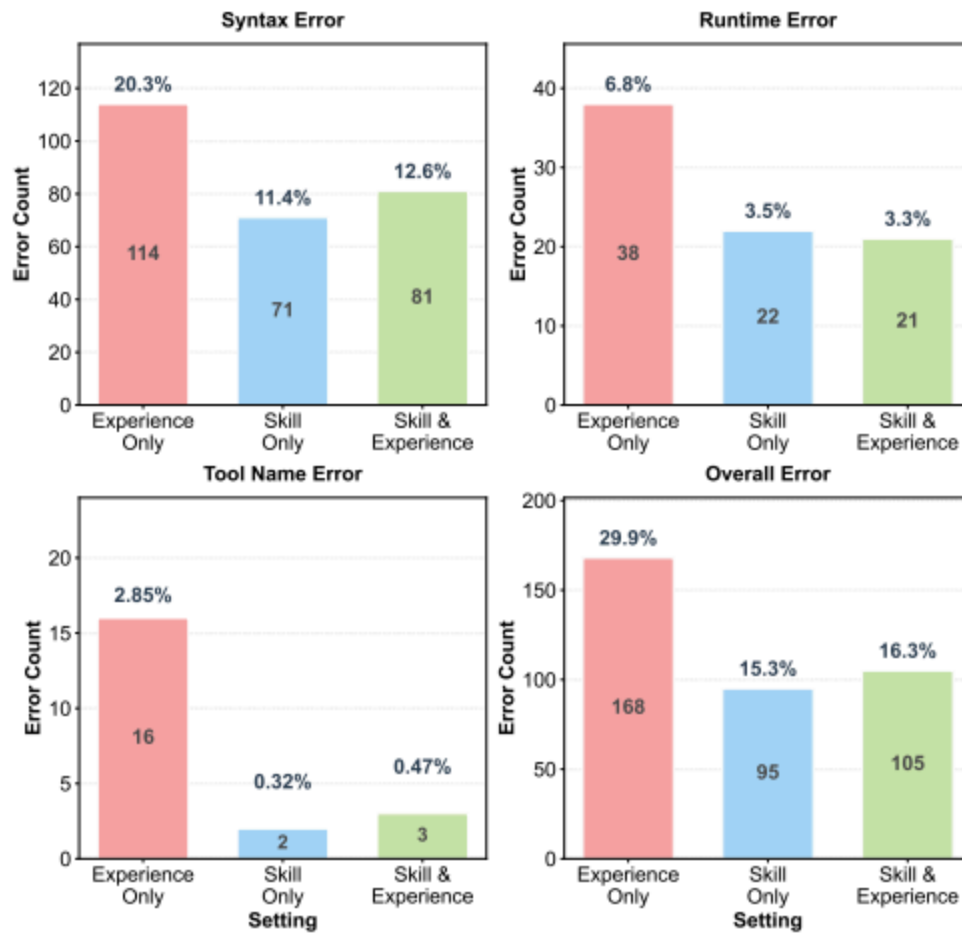
Setting	Average@4	Pass@4	Δ Avg@4
Ours - Full Pipeline	30.49	46.73	–
<i>Experience & Skill Ablation</i>			
w/o Experience	27.45	42.52	-3.04
w/o Skill	26.64	41.12	-3.85
<i>Phase 1 Ablation</i>			
w/o Experience Manager	26.40	42.06	-4.09
w/o Skill Manager	26.87	42.99	-3.62
<i>Phase 2 Ablation</i>			
w/o Task Decomposition	29.21	44.86	-1.28
w/o Task Adaptation	28.97	44.39	-1.52
w/ Tools	25.35	40.65	-5.14
No Tools	20.91	28.97	-9.58

（1）去掉 Skill 的下降稍大，表示在这个数据集上，结构化 Skill 的贡献略大；但不能据此断言所有任务中 Skill 都比 Experience 重要，因为这里只是在一个数据集和一个模型上做的消融

（2）阶段1：删除两个知识积累模块造成的下降最大，说明是否能从轨迹中提取并维护高质量知识，比检索阶段的小改进更加重要

（3）阶段2：两者都有贡献，但影响小于第一阶段的知识管理模块
知识本身的质量 > 知识的检索与适配技巧

工具执行错误分析：



Syntax Error: 语法错误

- ython 代码语法错误；
- JSON 或参数格式错误；
- 括号、引号缺失；
- 工具调用格式不符合要求

Runtime Error: 运行时错误

- 文件路径错误；
- 图片没有成功读取；
- 参数类型错误；
- 数组尺寸不一致；
- 对空搜索结果继续操作；
- 访问不存在的变量

Tool Name Error: 工具名称错误

- 把 `image_search` 写成另一个名字；
- 混淆 `web_search` 与 `visit`；

- 调用系统没有提供的图像处理工具

Overall Error: 总体错误

总体错误 = 语法错误 + 运行时错误 + 工具名称错误

工具调用比例

Setting	Code	Search-W	Search-I	Visit
<i>VisualToolBench</i>				
w/ Tools	66.63	31.70	–	1.66
Skill Only	65.96	31.10	–	2.82
Exp Only	74.49 (↑)	22.03 (↓)	–	2.56
Skill & Exp	76.97 (↑)	21.12 (↓)	–	1.58
<i>MMSearch-Plus</i>				
w/ Tools	6.18	71.07	15.43	7.32
Skill Only	7.94	67.07	17.87	7.12
Exp Only	13.21 (↑)	56.12 (↓)	24.63 (↑)	6.04
Skill & Exp	14.37 (↑)	55.08 (↓)	23.89 (↑)	5.66

VisualToolBench 中很多问题更适合直接使用代码处理图像或进行计算，而不是反复在网上搜索
遇到图片相关问题，应优先使用图片搜索和代码处理，而不是主要依赖通用网页搜索

在学习 Skill 和 Experience 时，采集多少条 rollout（尝试轨迹）会影响最终性能

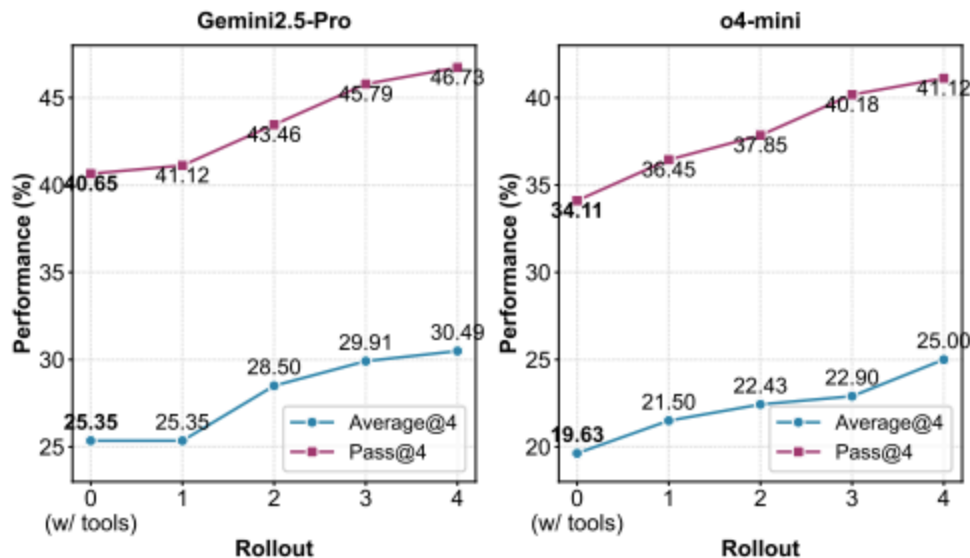
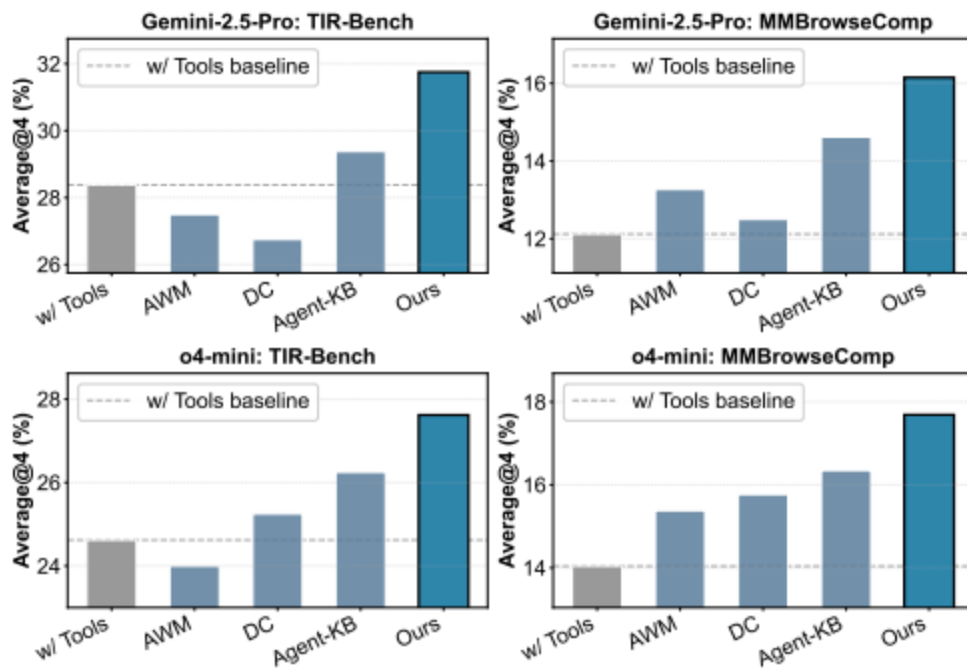


Figure 4. Performance comparison across different rollout values on VisualToolBench. Rollout $N = 0$ corresponds to the baseline with tools (w/ tools). The results show consistent improvement as the number of rollouts increases.

XSkill的学习效果会随着可用于归纳经验和技能的rollout次数增多而提高

跨任务迁移实验，考察从一个数据集学到的 Skill 和 Experience，换到一个没训练过的新数据集后是否仍然有效



VisualToolBench中学到的知识 → TIR-Bench
MMSearch-Plus中学到的知识 → MMBrowseComp